

A Score for Robot Helpers When Tasks Have Multiple Acceptable Time Windows

Metric side quest — AURA evaluation

May 5, 2026

Abstract

We are building a robot assistant that decides *when* to step in and help a person with a task. To check whether the robot does the right thing at the right time, we compare its decisions against a recording of when a human teleoperator chose to step in — the “ground truth”. Our old comparison method assumes there is exactly one correct moment for each helping action. In reality, many helping actions could be done in any of several time windows. This note describes a new ground-truth format and a new score that allows for those multiple correct windows, and shows that the new score behaves sensibly on a small worked example and on our existing hand-layup recording.

1 Why we need a new score

Imagine you are watching a robot help a person make tea. The person is the main worker; the robot only steps in for a few specific subtasks (say, “hand over the kettle”, “open the cupboard”). To check whether the robot’s timing is good, we wrote down a list of the moments when an expert teleoperator chose to step in, and we score the robot by comparing its actions to this list.

Up to now, each entry in that list looked like this:

“hand over the kettle should happen between 10 s and 15 s.”

Just one window per subtask. This works, but it lies in two ways:

Lie 1: there is often more than one correct window. “Hand over the kettle” could be done before the person fills the teapot, *or* during a brief pause while they wait for the water to boil. Both are reasonable. If the robot picks the second window and our list only mentions the first, we wrongly mark it as wrong.

Lie 2: a subtask might need to happen in two parts. “Bring the resin and then put it back” is one logical subtask, but the robot has to be in two different places at two different times. A single window cannot describe that.

We need a richer description of “when is it OK to do this”. Section 2 introduces such a description, and Section 3 introduces a score that uses it.

2 The new ground-truth format

We give each subtask a small tree of acceptable schedules:

- A **phase** is a single time window, e.g. “between 10 s and 15 s”. It is the smallest unit.
- An **option** is a list of phases that all have to happen for that option to count. “Bring the resin (between 5 s and 10 s) *and* put it back (between 25 s and 30 s)” is one option made of two phases.
- An **intervention** (one subtask) is a list of options. Only *one* option needs to happen, but every phase inside that one option has to happen. “Either bring the resin and put it back *or* bring the hardener and put it back” is one intervention with two options.

You can read it as: *an intervention is a choice of option, and an option is a list of windows that all have to be filled*. If we write it out symbolically, it looks like this:

$$\underbrace{\bigvee_{k=1}^{M_i}}_{\text{pick \textbf{one} option}} \quad \underbrace{\bigwedge_{j=1}^{L_{i,k}}}_{\text{satisfy \textbf{every} phase in it}} \quad \underbrace{t_{i,k,j} \in [s_{i,k,j}, e_{i,k,j}]}_{\text{within this window}}. \quad (1)$$

Here i indexes the subtask, k the option, j the phase. The “ $\vee$ ” means *or*; the “ $\wedge$ ” means *and*. Don’t worry too much about the symbols — the important point is “one option, but all its phases”.

2.1 Same idea, in JSON

The format on disk just mirrors the structure above. Listing 1 is the small example we will use throughout this note. Three subtasks, two options each, and some options have two phases.

Listing 1: Our toy ground truth: three subtasks, two options each. Notice how some windows are reused across different options on purpose.

```
"interventions": [
  {"id": "task_1",
   "options": [
     {"phases": [{"t_start": 5, "t_end": 10},
                  {"t_start": 25, "t_end": 30}]},
     {"phases": [{"t_start": 10, "t_end": 15},
                  {"t_start": 35, "t_end": 40}]}
   ]},
  {"id": "task_2",
   "options": [
     {"phases": [{"t_start": 0, "t_end": 5},
                  {"t_start": 10, "t_end": 15}]},
     {"phases": [{"t_start": 10, "t_end": 15},
                  {"t_start": 25, "t_end": 30}]}
   ]},
  {"id": "task_3",
   "options": [
     {"phases": [{"t_start": 0, "t_end": 5}]},
     {"phases": [{"t_start": 35, "t_end": 40}]}
   ]}
]
```

2.2 Why the example is interesting

The robot can only do one thing at a time. When we lay all the windows on a single timeline, we see that some choices clash:

Worked example

Suppose the robot picks “option A” for every subtask. Then:

- task 1 wants $[5, 10]$ and $[25, 30]$.
- task 2 wants $[0, 5]$ and $[10, 15]$.
- task 3 wants $[0, 5]$.

But task 2 and task 3 both want $[0, 5]$ — the robot can’t be in two places at once! So picking “option A everywhere” is impossible. The *only* fully-feasible plan is “A, A, B” (task 1 option A, task 2 option A, task 3 option B), which uses windows $\{[5, 10], [25, 30]\}$, $\{[0, 5], [10, 15]\}$, and $\{[35, 40]\}$ — none of these clash.

This is the key behaviour we want our score to capture: *the robot has to pick options that fit together*, not just options that individually look fine.

2.3 The old format still works

Our hand-layup recording was written in the old format (one window per subtask). The new format is a strict extension — we can read an old file and pretend each subtask has “one option containing one phase”. On such a file, the new score behaves like the old one. Nothing breaks.

3 The new score

We compare the robot’s predictions against the ground truth on four separate dimensions, then combine them. Each dimension is a number between 0 (terrible) and 1 (perfect).

$$A_{\text{score}} = w_{\text{disj}}A_{\text{disj}} + w_{\text{act}}A_{\text{act}} + w_{\text{cons}}A_{\text{cons}} + w_{\text{nec}}A_{\text{nec}}. \quad (2)$$

We default to weights 0.4, 0.2, 0.2, 0.2 — coverage matters slightly more than the rest, and the four weights add up to 1 so the final score also lies between 0 and 1. The four pieces are explained below; informally,

A_{disj} (**coverage**) — did the robot fill the required time windows?

A_{act} (**action**) — did it do the right *kind* of action in those windows?

A_{cons} (**consistency**) — do its choices fit together, or do they clash with the one-thing-at-a-time constraint?

A_{nec} (**necessity**) — did it act *only* when asked, or also a lot of useless extra times?

3.1 Background on overlap (IoU)

Most of the dimensions need a way to ask “how much do these two time windows overlap?”. The standard answer is the *intersection-over-union* (IoU):

$$\text{IoU}([a, b], [c, d]) = \frac{\text{length of overlap}}{\text{length of the union}}.$$

IoU is 1 when the two windows are exactly the same and 0 when they do not overlap at all.

Worked example

$\text{IoU}([5, 10], [5, 10]) = 1$ (identical). $\text{IoU}([5, 10], [7, 12]) = 3/7 \approx 0.43$ (they share $[7, 10]$, length 3, and together they cover $[5, 12]$, length 7). $\text{IoU}([5, 10], [20, 25]) = 0$ (no overlap at all).

3.2 A_{disj} : did the robot fill the right windows?

For each subtask we look only at predictions whose action label matches that subtask. Then for *each option* of that subtask we ask: how well does the robot’s best matching prediction fill each phase of this option, on average?

The option with the highest score wins, and that score is the subtask’s contribution to A_{disj} . The overall A_{disj} is just the average across all subtasks.

Worked example

Suppose for task 1, the robot predicted a “task 1”-labelled action in $[5, 10]$ and another in $[25, 30]$. Then:

- Option A has phases $[5, 10]$ and $[25, 30]$. Best IoU per phase is 1.0 and 1.0. Mean = 1.0.
- Option B has phases $[10, 15]$ and $[35, 40]$. Best IoU per phase is 0 and 0. Mean = 0.

Best option is A with score 1.0. So task 1 contributes 1.0 to A_{disj} , and we say the robot “selected option A” for task 1.

So A_{disj} rewards proposals that land inside one of the allowed schedules, and ignores how the robot describes its plan in words — we infer which option it meant from *when* it acted.

3.3 A_{act} : did it do the right kind of action?

For each prediction, we find the GT phase that overlaps it the most (highest IoU). If the robot’s action label matches the action label of that phase, the prediction is “correct”. A_{act} is the fraction of overlapping predictions that are correct.

Tie-breaking matters. Sometimes two phases from *different* subtasks share the same window (e.g. task 1 option B’s first phase and task 2 option A’s second phase are both $[10, 15]$). If the robot’s prediction at $[10, 15]$ is labelled “task 2”, we should not penalise it just because task 1 also *could* use that window. So when several phases tie at the highest IoU, we count the prediction as correct if any of them shares its action label.

3.4 A_{cons} : do the chosen options fit together?

Recall that the robot can only do one thing at a time. After we have inferred which option each subtask used (from A_{disj}), we check whether any pair of those chosen options collides — i.e. has phases overlapping in time.

We use the worst pairwise IoU among phases from *different* subtasks as the penalty:

$$A_{\text{cons}} = 1 - (\text{worst pairwise IoU between selected phases of different subtasks}).$$

If no two selected phases overlap, $A_{\text{cons}} = 1$. If two of them overlap exactly, $A_{\text{cons}} = 0$.

Worked example

The toy GT has exactly one fully-feasible combination: A, A, B (task 1 option A, task 2 option A, task 3 option B). On that combination, no pair of selected phases overlaps and $A_{\text{cons}} = 1$. On the combination A, A, A (which is what the “always pick option 0” adversary chooses), task 2 phase $[0, 5]$ and task 3 phase $[0, 5]$ are identical, so the worst pair has IoU 1 and $A_{\text{cons}} = 0$.

This is the dimension that the old score could not see at all.

3.5 A_{nec} : did it intervene only when asked?

Take *all* the phase windows in the GT (across every option of every subtask) and union them into one big set of allowed time Φ . Then ask: out of the total time the robot spent acting, how much of it was inside Φ ?

$$A_{\text{nec}} = \frac{\text{total prediction time spent inside } \Phi}{\text{total prediction time}}.$$

A robot that fires all over the place gets low A_{nec} . A robot that acts only inside legitimate windows gets $A_{\text{nec}} = 1$. By convention, $A_{\text{nec}} = 1$ when the robot makes no predictions at all (you cannot make a false alarm if you say nothing — a silent robot is punished by A_{disj} instead).

4 The adversaries

To check whether the score actually distinguishes good behaviour from bad behaviour, we wrote thirteen *adversaries* — fake robots each designed to fail in one specific way. The full list is in Table 1.

Adversary	What it does
Oracle (consistent)	Picks an option for each subtask such that no two chosen phases clash. Fires exactly those phases. <i>Should score perfectly.</i>
Oracle (first option)	Always picks option index 0. Same correct <i>timing</i> , but its choices may clash.
Conflict-prone	On purpose picks options whose phases overlap as much as possible.
Phase-collapsed	Like the consistent oracle, but only fires the <i>first</i> phase of each option.
Half-coverage	Like the consistent oracle, but covers only every other subtask.
Wrong-skill	Right windows, but every prediction’s label is shifted to the <i>next</i> subtask’s label.
Near-miss timing	Consistent oracle shifted by +1.5 s.
Far-shifted	Consistent oracle shifted by +8 s.
Premature firer	Consistent oracle shifted by −1.5 s.
Over-extender	Right starts; every phase end is padded by +4 s.
One-shot blanket	A single prediction covering the whole task duration.
Spammer	Rapid-fire short predictions of the most-common action label.
Silent	Empty prediction track — never acts.

Table 1: The thirteen adversaries used to validate the new score.

5 Results

5.1 Toy three-task example

Figure 1 stacks the GT (top three rows) above each adversary’s predictions. Table 2 gives the four-dimension breakdown.

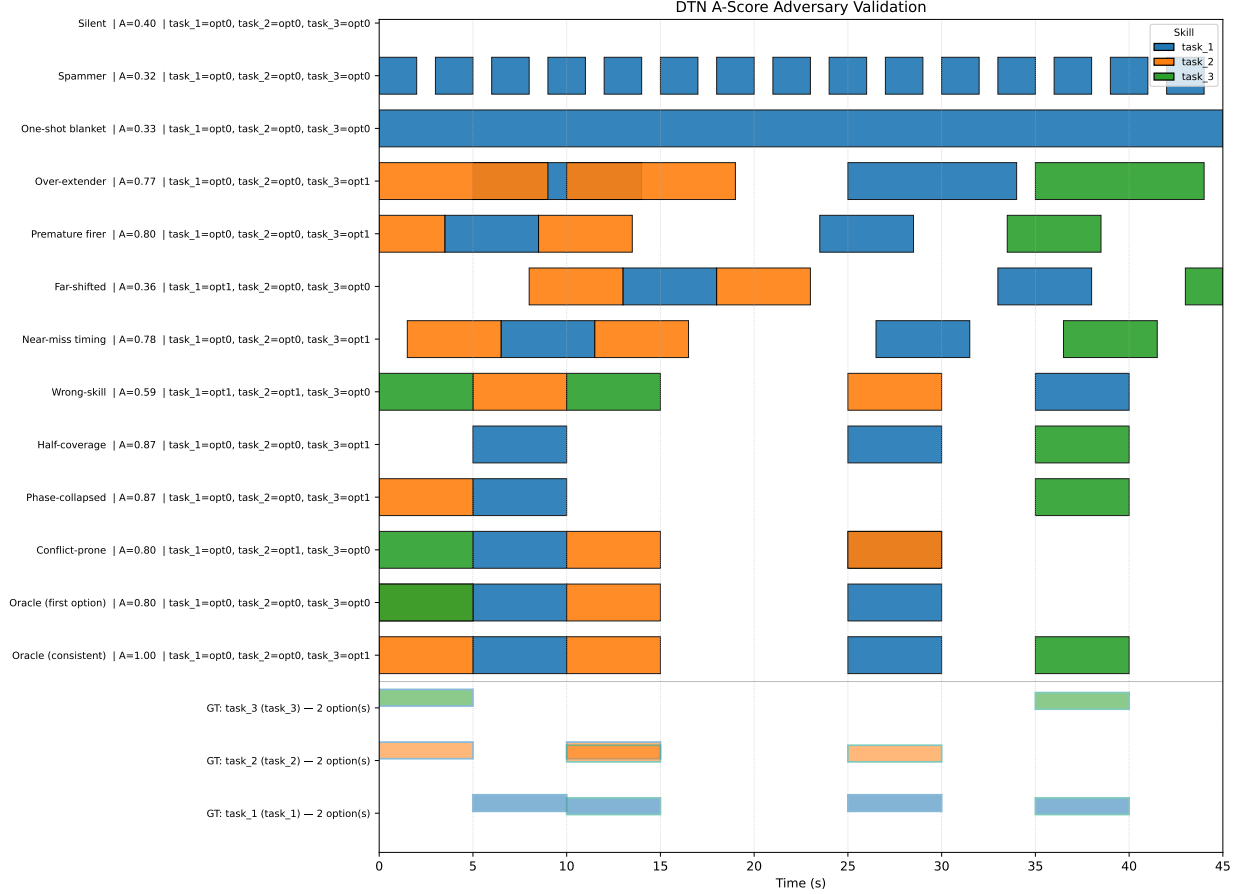


Figure 1: Toy DTN. Top three rows: the ground truth, with each subtask’s two options drawn one above the other (different edge colours = different options). Lower rows: each adversary’s predictions. The label on each row reports the final score and the option the metric *inferred* that the adversary was aiming at.

What the table tells us. The four dimensions each surface a different kind of failure:

- **Coverage vs. consistency.** “Oracle (first option)” and “Conflict-prone” fill every required window with the right action label, so $A_{\text{disj}} = 1$ and $A_{\text{act}} = 1$. But the options they pick overlap each other, so A_{cons} drops to 0. They end up below the consistent oracle. *This is the failure mode the old score could not see.*
- **Coverage vs. correctness.** “Wrong-skill” has the right windows but the wrong labels, which both lowers A_{act} and confuses the option-selection logic, dragging A_{cons} down too.

Adversary	A_{disj}	A_{act}	A_{cons}	A_{nec}	A_{score}
Oracle (consistent)	1.00	1.00	1.00	1.00	1.00
Oracle (first option)	1.00	1.00	0.00	1.00	0.80
Conflict-prone	1.00	1.00	0.00	1.00	0.80
Phase-collapsed	0.67	1.00	1.00	1.00	0.87
Half-coverage	0.67	1.00	1.00	1.00	0.87
Wrong-skill	0.67	0.60	0.00	1.00	0.59
Near-miss timing	0.54	1.00	1.00	0.82	0.78
Far-shifted	0.19	1.00	0.00	0.45	0.37
Premature firer	0.57	1.00	1.00	0.87	0.80
Over-extender	0.56	1.00	1.00	0.73	0.77
One-shot blanket	0.04	1.00	0.00	0.56	0.33
Spammer	0.13	0.78	0.00	0.53	0.32
Silent	0.00	1.00	0.00	1.00	0.40

Table 2: Score breakdown on the toy DTN (weights $w = (0.40, 0.20, 0.20, 0.20)$).

- **Coverage vs. necessity.** “Spammer” and “one-shot blanket” flood the timeline; their A_{nec} correctly drops below 0.6 because most of their predicted time lies outside any legitimate window.
- **Silent strategy.** The silent adversary scores $A_{\text{score}} = 0.4$ — not zero, because by convention the “correctness” and “necessity” axes give credit for not making bad predictions. But $A_{\text{disj}} = 0$ keeps it well below any adversary that actually does something useful.

5.2 Hand-layup recording (old-format ground truth)

Our existing hand-layup ground truth has only one option per subtask and one phase per option (the old format). On this kind of input, A_{cons} is automatically 1 (you cannot create a clash if there is only one possible choice), and A_{disj} becomes plain mean phase IoU — exactly the same as the old timing axis. Figure 2 and Table 3 confirm this: the relative ranking of adversaries on the hand-layup recording matches what we would expect from the old score.

Adversary	A_{disj}	A_{act}	A_{cons}	A_{nec}	A_{score}
Oracle (consistent)	1.00	1.00	1.00	1.00	1.00
Oracle (first option)	1.00	1.00	1.00	1.00	1.00
Conflict-prone	1.00	1.00	1.00	1.00	1.00
Phase-collapsed	1.00	1.00	1.00	1.00	1.00
Half-coverage	0.50	1.00	1.00	1.00	0.80
Wrong-skill	0.00	0.00	1.00	1.00	0.40
Near-miss timing	0.94	1.00	1.00	0.98	0.97
Far-shifted	0.70	1.00	1.00	0.90	0.86
Premature firer	0.94	1.00	1.00	0.98	0.97
Over-extender	0.92	1.00	1.00	0.94	0.96
One-shot blanket	0.04	0.00	1.00	0.70	0.36
Spammer	0.01	0.25	1.00	0.70	0.39
Silent	0.00	1.00	1.00	1.00	0.60

Table 3: Score breakdown on the hand-layup ground truth (old format, one option / one phase per subtask).

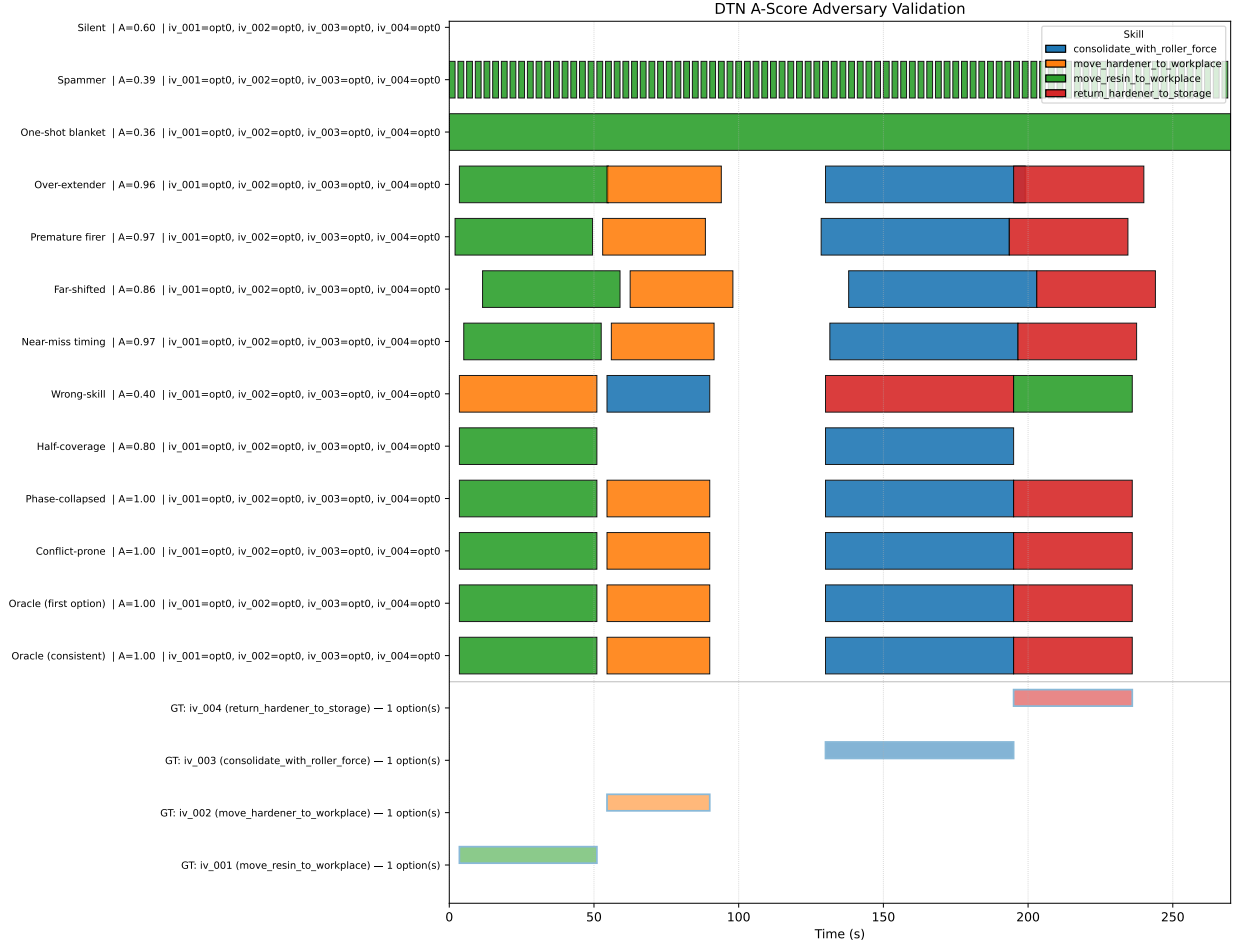


Figure 2: Hand-layup ground truth (old format, one option / one phase per subtask). With nothing to choose between, $A_{\text{cons}} = 1$ for every adversary and the score reduces to a coverage-plus-action-plus-necessity blend.

6 Discussion

6.1 Why A_{cons} is not just yes/no

We could imagine a much harsher version of A_{cons} : 1 if the chosen options fit together perfectly, 0 if there is any clash at all. We chose a smoother version (1 minus the worst pair-IoU) because a strategy that overlaps two phases by 1% is qualitatively different from one that fully collides on a 5-second window. The smooth version still gives 1.0 for a feasible plan and 0.0 for a fully-conflicting one — but a near-miss is mostly fine.

6.2 Why the silent strategy looks weird on the toy DTN

On the toy DTN, the silent adversary scores $A_{\text{cons}} = 0$ rather than the intuitive 1. This happens because, with no predictions to look at, the option-selection logic falls back to “option 0 for everything”, and on the toy GT those options happen to clash. The silent adversary is still ranked very low overall ($A_{\text{score}} = 0.4$) because $A_{\text{disj}} = 0$ dominates, so this is not a real bug — but it is a small wart that we could clean up by leaving A_{cons} undefined when no predictions match.

6.3 Connection to the formal literature

This work was inspired by Osanlou et al. [1], who study *Disjunctive Temporal Networks with Uncertainty* and ask whether a strategy exists that can satisfy all constraints no matter how the world unfolds. We are doing something simpler: we already have a fixed prediction track and we just want to score it. We borrow their constraint shape (equation 1) but not their tree-search algorithm.

7 Reproducibility

The code lives in `aura/scripts/eval/metric_side_quest/`:

- `disjunctive_gt.py` — the new ground-truth dataclasses and a loader that also accepts old-format files.
- `dtm_score.py` — the four scoring axes plus a small backtracking solver used by the consistent-oracle adversary.
- `dtm_adversaries.py` — the thirteen adversaries and the command-line interface.
- `toy_dtm_three_tasks.robot_gt.json` — the toy ground truth used in this note.
- `layup_gesture_demo_stationary_with_overlay.robot_gt.v2.json` — new-format copy of the hand-layup ground truth (old-format file unchanged).

To regenerate the figures and tables in this document:

```
# Toy DTN (Figure 1, Table 2)
python scripts/eval/metric_side_quest/dtm_adversaries.py

# Hand-layup recording (Figure 2, Table 3)
python scripts/eval/metric_side_quest/dtm_adversaries.py \
  --gt scripts/eval/metric_side_quest/layup_gesture_demo_stationary_with_overlay.
    robot_gt.v2.json \
  --output scripts/eval/metric_side_quest/out/fig_dtm_adversaries_layup.pdf
```

8 Conclusion

We extended our robot-helper score to handle subtasks that admit more than one acceptable schedule. The key idea is to describe each subtask as a small “or” over options, where each option is itself an “and” over time windows. The resulting score has four dimensions — coverage, action correctness, schedule consistency, and necessity — and the new *consistency* dimension is what captures the failure mode of picking options that clash on a single-resource agent. We checked the score against thirteen synthetic adversaries on a toy three-subtask example and on our existing hand-layup recording. It ranks them sensibly and reduces to the old behaviour when the ground truth is in the old format. The next step is to plug it into the existing evaluation pipeline.

References

- [1] K. Osanlou, J. Frank, A. Bursuc, T. Cazenave, E. Jacopin, C. Guettier, and J. Benton. Solving Disjunctive Temporal Networks with Uncertainty under Restricted Time-Based Controllability using Tree Search and Graph Neural Networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022.